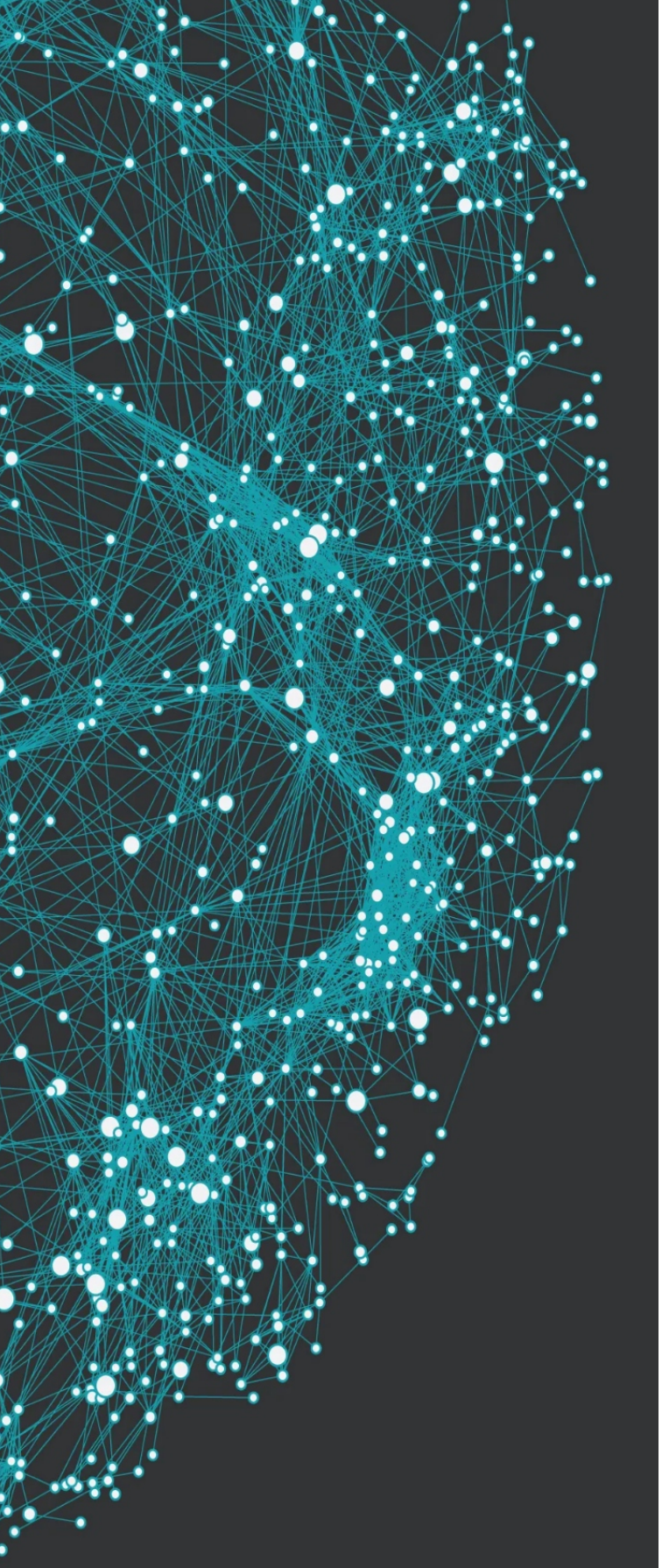


ESIROI
ÉCOLE D'INGÉNIEURS
UNIVERSITÉ DE LA RÉUNION

UNIVERSITÉ
DE LA RÉUNION

Velib Data Flow : Ingestion, Stockage et Analyse sur Cluster Distribué

RAPPORT

Challenge données massive
Mathilde LEPETIT

Date
28 Décembre 2025

Rédiger par
Lova Fenohasina RAFANOMEZANTSOA
Ny Hasina Marolahy VAGNO

4A Informatique – Promotion 2027

Table des matières

Introduction **1**

1 Infrastructure distribuée et "Bare Metal" virtuel **2**
1.1 Architecture Physique et Logique 2
1.2 Choix Technologiques et Stratégie de Déploiement 3
1.3 Déploiement Automatisé et Validation 4
1.4 Challenges Techniques et Solutions (Retour d'Expérience) 7

2 Prétraitement des Données **8**
2.1 Sources de Données et Collecte 8
2.2 Métriques d'Évaluation de la Qualité des Données 9
2.3 Pipeline de Prétraitement 10
2.4 Ingénierie des Caractéristiques 10

3 Stratégie d'Évaluation et de Comparaison des Modèles **11**
3.1 Protocole de Validation Croisée Temporelle 11
3.2 Benchmark des Algorithmes 12

4 Bilan des difficultés rencontrées et solutions apportées **14**
4.1 Data Leakage 14
4.2 Complexité du problème réel 14
4.3 Volumétrie et hétérogénéité des données 15
4.4 Choix et évaluation des modèles 15

5 Conclusions et recommandations **15**
5.1 Synthèse des apprentissages 15
5.2 Évaluation du modèle final 15
5.3 Pistes d'amélioration 15

Conclusion **17**

Table des figures

1 Architecture de l'infra 3
2 Resultat d'installation - trois datanode 6
3 Resultat d'installation - trois node 7
4 Taux de remplissage moyenne (semaine vs weekend) 11
5 Audit de data leakage 13

Introduction

À l'ère de la "Smart City", la gestion optimisée de la mobilité urbaine repose désormais sur la capacité à transformer des flux de données massifs en informations décisionnelles. Le projet "Challenge Données Massives", réalisé dans le cadre du cursus ESIROI 4A, s'inscrit dans cette dynamique en nous confrontant à la réalité brute de l'ingénierie des données : comment construire une chaîne de valeur complète, de l'infrastructure physique à la prédiction intelligente, avec des ressources matérielles contraintes ?

L'objectif de ce projet est double. D'une part, il s'agit de relever un défi d'infrastructure : déployer un cluster de calcul distribué (Hadoop/Spark) non pas sur des serveurs Cloud élastiques, mais sur une architecture "Bare Metal" simulée à l'aide de deux machines physiques interconnectées. D'autre part, le projet vise un objectif analytique : exploiter cette puissance de calcul pour prédire la disponibilité des vélos en libre-service (Vélib') à Paris, en croisant des historiques d'utilisation et des données météorologiques.

Ce rapport retrace notre démarche d'ingénierie qui a consisté à fusionner les casquettes d'Administrateur Système et de Data Scientist. La **première partie** détaille la conception de l'infrastructure distribuée. Nous y exposons comment, face aux restrictions réseaux du campus et aux limites de mémoire vive, nous avons orchestré le déploiement de quatre nœuds virtuels via **Ansible**, garantissant reproductibilité et performance. La **deuxième partie** aborde l'ingestion et le traitement des données (ETL) avec **Apache Spark**. Nous verrons comment le nettoyage et l'ingénierie des fonctionnalités (Feature Engineering) ont transformé des millions de logs bruts en un jeu de données exploitable. Enfin, la **troisième partie** présente la modélisation prédictive. Nous analyserons les performances de notre modèle XGBoost optimisé et reviendrons sur un cas d'école de "Data Leakage" qui a faussé nos premiers résultats, offrant une leçon méthodologique précieuse sur la rigueur scientifique nécessaire en Machine Learning.

Au-delà de la simple exécution technique, ce rapport démontre notre capacité à maîtriser le cycle de vie complet de la donnée, des câbles réseaux physiques jusqu'aux hyper-paramètres de l'algorithme.

1 Infrastructure distribuée et “Bare Metal” virtuel

L’objectif de cette première phase n’était pas simplement d’instancier des services, mais de concevoir une architecture **résiliente, performante et reproductible**. Contraints par des ressources matérielles limitées et un environnement réseau académique restrictif (PlanetCampus), nous avons opté pour une approche hybride : un cluster distribué sur deux hôtes physiques interconnectés, orchestré par une logique *Infrastructure as Code* (IaC).

1.1 Architecture Physique et Logique

La construction d’un cluster Hadoop/Spark fonctionnel nécessite de garantir une haute disponibilité du NameNode (mémoire vive) et une capacité de traitement parallèle sur les DataNodes (CPU/Disque). Disposant de deux machines physiques aux capacités asymétriques, nous avons conçu une topologie en “grappe bipolaire”.

1.1.1 Topologie Matérielle et Réseau “Dual-Homed”

L’infrastructure repose sur deux unités centrales (PC1 et PC2) reliées physiquement par un câble Ethernet dédié. Cette configuration répond à une contrainte majeure : l’isolation des clients sur le réseau Wi-Fi du campus (“PlanetCampus”) empêche la communication inter-machines.

Pour contourner ce blocage tout en maintenant l’accès aux dépôts logiciels (Internet), nous avons implémenté une architecture réseau à double interface pour chaque Machine Virtuelle (VM) :

Interface NAT (ens33) : Permet l’accès sortant vers Internet via la carte Wi-Fi de l’hôte pour le téléchargement des paquets et les mises à jour système.

Interface Bridge (ens37) : Connectée à la carte Ethernet physique dédiée. Elle constitue le **réseau privé du cluster** (subnet `192.168.50.0/24`), garantissant une bande passante Gigabit dédiée au trafic HDFS (réplication de blocs) et Spark (Shuffle), sans interférence avec le trafic du campus.

1.1.2 Répartition des Rôles et Dimensionnement

Le cluster est composé de 4 nœuds virtuels. La répartition des ressources a été calculée pour maximiser l’usage de la RAM disponible sans provoquer de *swapping* sur les hôtes physiques, ce qui serait fatal aux performances de Java (Garbage Collector).

PC 2 (Hôte Secondaire) - Le Nœud Maître :

Ressources : 4 CPU / 16 Go RAM.

VM Master (`192.168.50.160`) : Ce nœud est le cœur du cluster. Doté de **12 Go de RAM**, il héberge les démons critiques : le **NameNode** (métadonnées HDFS) et le **ResourceManager** (YARN). Sa position isolée sur le PC2 lui garantit une moindre contention CPU face aux nœuds de calcul intensif. Il agit également comme NodeManager d’appoint.

PC 1 (Hôte Principal) - La Force de Calcul :

Ressources : 4 CPU / 32 Go RAM.

VMs Workers (`.161`, `.162`, `.163`) : Trois nœuds identiques configurés avec **8 Go de RAM** chacun. Ils hébergent les **DataNodes** (stockage) et les **NodeManagers** (exécuteurs Spark). Cette densification sur le PC1 permet de profiter de la localité des données pour les tâches distribuées sur ce même hôte.

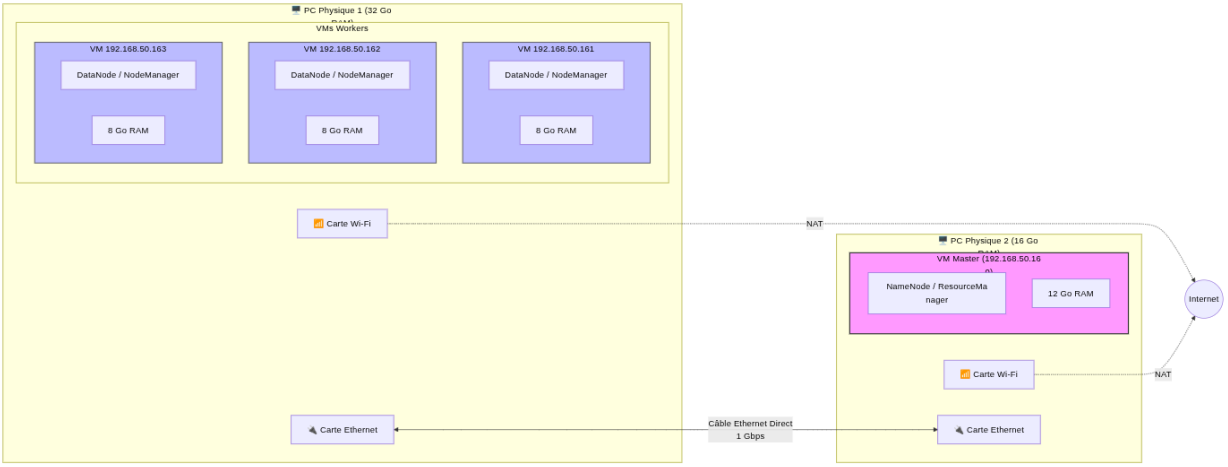


FIGURE 1 – Architecture de l'infra

Cette architecture nous permet d'obtenir un cluster totalisant **36 Go de RAM distribuée** et **4 cœurs physiques (16 vCPU)**, une puissance de feu suffisante pour traiter les datasets Vélib' sans recourir au Cloud.

1.2 Choix Technologiques et Stratégie de Déploiement

La réussite du projet reposait sur la sélection d'une stack technologique stable et d'une méthode de déploiement capable de gérer la complexité multi-hôtes décrite ci-dessus.

1.2.1 Virtualisation et OS

Contrairement à une approche conteneurisée (Docker), nous avons opté pour une virtualisation lourde via **VMware Workstation Pro**. Ce choix est motivé par deux facteurs :

Performance I/O : VMware offre une gestion des disques virtuels (`.vmdk`/`.vmx`) très performante et une isolation stricte des ressources, essentielle pour simuler un environnement "Bare Metal" réaliste.

Standard Industriel : Son utilisation simule fidèlement les infrastructures on-premise classiques.

Pour le Système d'Exploitation, notre choix s'est porté sur **Ubuntu 24.04 LTS**.

Stabilité du Noyau : La version 24.04 intègre un kernel Linux optimisé pour les nouvelles générations de processeurs et une gestion de la mémoire virtuelle améliorée.

Richesse des Dépôts : Contrairement à Debian qui nécessite souvent des configurations de dépôts tiers, Ubuntu propose nativement **OpenJDK 11** dans ses dépôts officiels. Java 11 a été retenu comme le point d'équilibre parfait : il est requis pour les versions récentes de Spark (3.x) tout en assurant une rétrocompatibilité totale avec Hadoop 3.

Concernant le Big Data, nous avons déployé les versions “State of the Art” :

Hadoop 3.4.2 : Dernière version stable, apportant des corrections majeures sur la gestion de l'Erasure Coding (optimisation stockage).

Spark 3.5.7 : Choisie pour sa compatibilité avec Java 11 et ses performances accrues sur l'optimiseur de requêtes Catalyst.

1.2.2 Stratégie de Déploiement : Le choix d'Ansible (IaC)

Le déploiement d'un cluster sur 4 machines réparties sur 2 hôtes physiques pose un défi de maintenance critique. Trois approches ont été évaluées :

Scripts Shell/Python (Rejeté) Bien que pédagogique, l'approche par scripts impératifs (Bash/Python) souffre d'un défaut majeur : le manque d'**idempotence**. Relancer un script interrompu risque de dupliquer des configurations ou de corrompre des fichiers XML Hadoop. De plus, la gestion des erreurs sur 4 nœuds simultanés devient rapidement ingérable.

Docker / Kubernetes (Rejeté) L'option Docker, bien que séduisante pour sa légèreté, a été écartée pour des raisons d'architecture réseau. Faire communiquer des conteneurs répartis sur deux hôtes physiques distincts (sans orchestrateur type Kubernetes ou Swarm) nécessite une configuration réseau complexe (Overlay Network, routing manuel). De plus, l'abstraction du système de fichier par Docker (OverlayFS) introduit une pénalité de performance (overhead) en écriture, préjudiciable pour les DataNodes HDFS qui nécessitent un accès disque direct et rapide.

Ansible (Retenu) Nous avons choisi **Ansible**, un outil d'automatisation sans agent (*agentless*), utilisant simplement SSH.

Idempotence : Ansible garantit que l'état final est conforme à la description, peu importe le nombre d'exécutions. C'est crucial lors des phases de “Trial & Error” de la configuration `hdfs-site.xml` ou `yarn-site.xml`.

Parallélisme : Ansible déploie simultanément sur les 4 nœuds (PC1 et PC2), réduisant le temps d'installation de plusieurs heures (manuel) à quelques minutes.

Gestion de l'Inventaire : Le fichier `hosts` d'Ansible nous permet de définir explicitement quel nœud est Master et quels nœuds sont Workers, appliquant dynamiquement les configurations appropriées.

C'est cette combinaison — **Virtualisation robuste (VMware) + Automatisation flexible (Ansible)** — qui nous a permis de transformer deux ordinateurs standards en un cluster Big Data cohérent et fonctionnel.

1.3 Déploiement Automatisé et Validation

L'installation manuelle d'un cluster Hadoop est une opération répétitive et sujette à l'erreur humaine (fautes de frappe dans les fichiers XML, oubli d'une variable d'environnement). Pour garantir un déploiement fiable et identique sur les 4 nœuds, nous avons structuré notre déploiement Ansible en quatre étapes séquentielles.

1.3.1 Séquencement du Playbook Ansible

Notre stratégie d'automatisation s'est divisée en rôles distincts pour maintenir une clarté modulaire :

Étape 1 : Préparation du Système (Socle Commun) Avant toute installation logicielle, il est impératif d'harmoniser les configurations système. Ce rôle Ansible assure :

— La mise à jour des paquets `apt` et l'installation des outils de base (`net-tools`, `vim`, `ssh`).
— La **désactivation de l'IPv6** et la configuration du `hostname` pour éviter les conflits de résolution DNS.
— L'échange des clés SSH : Le maître doit pouvoir se connecter aux workers sans mot de passe.
— La configuration du pare-feu (`ufw`) pour autoriser le trafic interne sur les ports critiques Hadoop (9000, 9870, 8088).

Étape 2 : Installation de Java 11 (Le Runtime) Java est le moteur d'Hadoop. Le playbook installe `openjdk-11-jdk` et configure dynamiquement la variable `$JAVA_HOME` dans le `.bashrc` de l'utilisateur `hadoop`. Cette étape est critique : une mauvaise variable d'environnement empêche le démarrage immédiat des démons.

Étape 3 : Déploiement de Hadoop 3.4.2 C'est l'étape la plus complexe. Ansible télécharge l'archive binaire, l'extrait dans `/opt/hadoop`, et surtout, déploie les fichiers de configuration via des **templates Jinja2**.

— *L'avantage des templates :* Plutôt que de copier des fichiers statiques, nous utilisons des variables. Par exemple, dans `core-site.xml`, l'adresse du NameNode est injectée dynamiquement : `<value>hdfs://{{ namenode_ip }}:9000</value>`. Cela rend le script réutilisable même si l'IP du maître change.

Étape 4 : Déploiement de Spark 3.5.7 Similaire à Hadoop, cette étape installe Spark et configure `spark-defaults.conf` pour qu'il pointe vers YARN comme gestionnaire de ressources (`spark.master=yarn`).

1.3.2 Zoom sur le Code

Pour concrétiser l'intérêt d'Ansible, prenons l'exemple critique de la résolution de noms au sein du cluster. Afin que le *NameNode* et les *DataNodes* communiquent efficacement sans serveur DNS dédié, nous devons maintenir un fichier `/etc/hosts` synchronisé sur toutes les machines.

Plutôt que de copier un fichier statique (ingérable à grande échelle), nous utilisons le moteur de template **Jinja2** pour le générer dynamiquement.

Le modèle dynamique (`templates/hosts.j2`) : Ce fichier agit comme un squelette. Il utilise une boucle `for` pour itérer sur l'ensemble du groupe `all` défini dans l'inventaire.

```
127.0.0.1   localhost
# Génération dynamique basée sur l'inventaire Ansible
{% for server in groups['all'] %}
{{ hostvars[server]['ansible_host'] }}  {{ server }}
{% endfor %}
```

La tâche d'application (`playbook.yml`) : Cette tâche déploie le résultat généré sur l'ensemble du parc.

```
- name: Déployer la configuration réseau (/etc/hosts)
  template:
    src: templates/hosts.j2
    dest: /etc/hosts
    owner: root
    mode: '0644'
```

Analyse technique : Cet extrait met en lumière deux mécanismes fondamentaux de notre stratégie de déploiement :

— **L'abstraction par les variables (Facts) :** Le script accède aux métadonnées de chaque hôte via `hostvars`. Ansible récupère l'adresse IP réelle (`ansible_host`) et le nom d'hôte associés dans l'inventaire, sans que nous ayons à les coder en dur dans le script.

— **La scalabilité horizontale (Scalability) :** L'ajout de nouveaux *Workers* au cluster devient trivial. Il suffit de déclarer les nouvelles machines dans l'inventaire et de relancer le playbook. Ansible mettra à jour simultanément le fichier `/etc/hosts` sur l'intégralité du parc, qu'il contienne 4 ou 50 machines, sans modifier une seule ligne de code.

— **L'idempotence :** Avant d'appliquer la modification, Ansible compare le fichier généré avec celui présent sur le disque. Si le contenu est identique, aucune action n'est effectuée (état "OK"). Cette propriété garantit la stabilité du cluster en évitant des réécritures inutiles et permet de relancer le déploiement à volonté sans risque de corruption.

1.3.3 Validation Technique

Une fois le playbook exécuté (temps moyen : 8 minutes), nous avons procédé à la validation des services.

Validation HDFS (Stockage) : La commande `jps` sur le maître confirme la présence du `NameNode` et du `ResourceManager`. L'interface web (Port 9870) est le juge de paix : elle doit afficher **"Live Nodes : 3"** (ou 4 selon la config). Si un nœud manque, c'est souvent un problème de `clusterID` divergent entre le NameNode et le DataNode, nécessitant un nettoyage des dossiers `/tmp`.

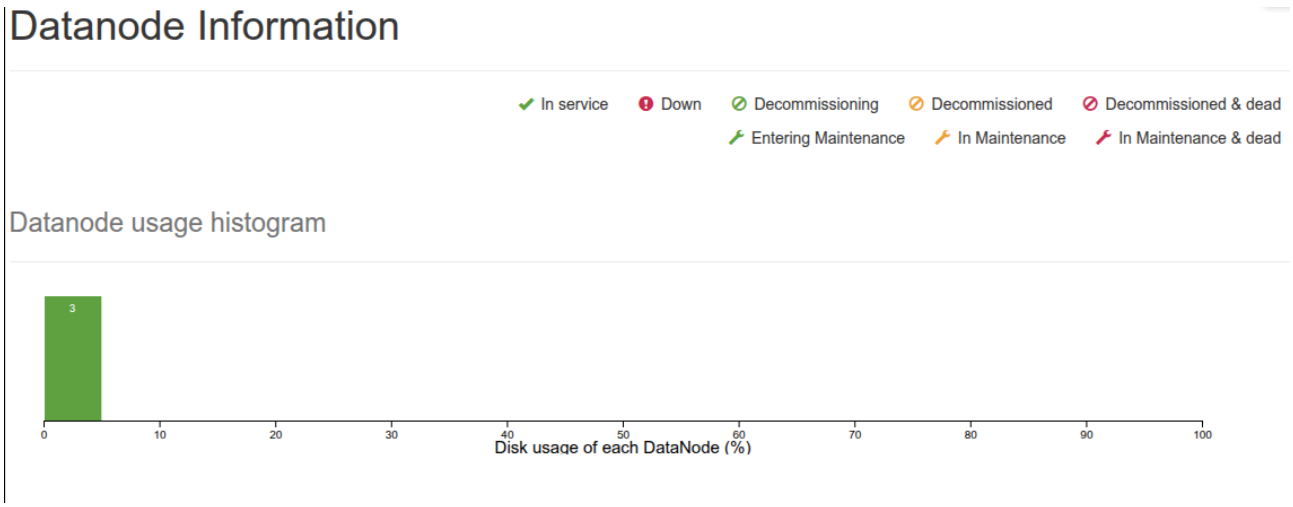


FIGURE 2 – Resultat d'installation - trois datanode

Validation YARN (Calcul) : L'interface de YARN (Port 8088) nous montre les `NodeManagers` actifs. Pour valider la capacité de calcul, nous avons soumis un job exemple (le calcul de Pi) : `spark-submit --class org.apache.spark.examples.SparkPi ...` Le succès de ce job confirme que Spark arrive à négocier des conteneurs auprès de YARN sur les machines distantes.

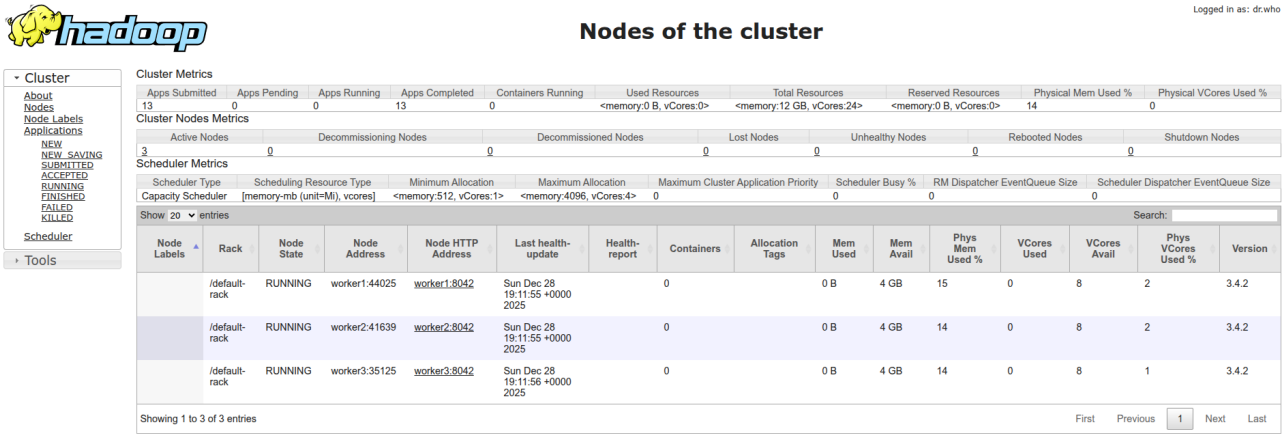


FIGURE 3 – Resultat d'installation - trois node

1.4 Challenges Techniques et Solutions (Retour d'Expérience)

La mise en place de ce cluster "Bare Metal" simulé a révélé des difficultés que l'usage de solutions Cloud clé-en-main (AWS EMR, Databricks) masque habituellement.

1.4.1 La Contrainte des Ressources (Le "Tetris" Mémoire)

Le Problème : Nos hôtes physiques ne sont pas des serveurs de production. Avec 16 Go de RAM sur le PC2, allouer 12 Go à la VM Master laissait très peu de marge pour l'OS hôte (Windows/Linux). Lors des premiers tests, le PC2 "gelait" à cause du swapping intensif.

La Solution : Nous avons affiné les paramètres de la JVM (Java Virtual Machine) dans `hadoop-env.sh`. Nous avons limité le *Heap Size* des démons non-critiques (comme le SecondaryNameNode) pour réserver la mémoire aux tâches Spark. De plus, nous avons désactivé l'interface graphique (GUI) sur les VMs Ubuntu Server pour gagner environ 600 Mo de RAM par VM.

1.4.2 Le Casse-tête Réseau "PlanetCampus"

Le Problème : Le réseau de l'université (PlanetCampus) applique une isolation stricte des clients (AP Isolation). Impossible pour nos VMs de se pinger entre elles via le Wi-Fi, rendant le cluster inopérant.

La Solution : C'est ici que l'architecture à double carte réseau (décrite en 1.1) a été décisive.

1. Nous avons câblé physiquement les deux PC.
2. Nous avons configuré les VMs en mode "Pont" (Bridge) sur cette interface Ethernet.
3. Cependant, les VMs sur le PC1 ne trouvaient pas les VMs sur le PC2 par leur nom d'hôte. La résolution DNS échouait.
4. Nous avons dû utiliser Ansible pour écrire en dur les correspondances IP/Nom dans le fichier `/etc/hosts` de **chaque** machine du cluster. Sans cela, Hadoop refusait de démarrer, car il se base sur les noms d'hôtes pour identifier les DataNodes.

1.4.3 L'Absence de Support "Out-of-the-Box"

Le Problème : Contrairement aux TPs guidés, nous n'avions pas de scripts Ansible fournis. La documentation officielle Apache Hadoop est exhaustive mais très fragmentée.

La Solution : Nous avons adopté une approche itérative. Nous avons d'abord installé le cluster manuellement sur 2 VMs pour comprendre les fichiers de config (`core-site.xml`, `hdfs-site.xml`). Une fois la configuration valide identifiée, nous l'avons traduite en templates Ansible. Cela nous a permis de comprendre *ce que* nous automatisons, plutôt que d'exécuter un script "boîte noire".

1.4.4 Bilan

Cette phase d'infrastructure, bien que chronophage, nous a permis de construire un socle robuste. Nous disposons désormais d'un cluster fonctionnel, monitoré et capable d'exécuter des calculs distribués, prêt pour l'ingestion des données Vélib' en Phase 2. L'usage d'Ansible transforme une contrainte de déploiement complexe en un atout de maintenabilité.

2 Prétraitement des Données

2.1 Sources de Données et Collecte

Les données utilisées proviennent de plusieurs sources hétérogènes, tant par leur format que par leur granularité temporelle. Le tableau suivant synthétise les principales sources mobilisées :

Source	Période	Volume	Variables Clés
Github Vélib' (Parquet)	Octobre–Décembre 2024	1 414 fichiers	`numbikesavailable`, `numdocksavailable`, `capacity`
API Vélib' (JSON)	Décembre 2025	110 fichiers	Variables identiques
Open-Meteo	Octobre 2024 – Décembre 2025	2 fichiers CSV	Variables météo horaires
Éducation Nationale	Calendrier scolaire	API	Jours de vacances

La collecte a été réalisée via des appels API automatisés et des téléchargements de fichiers pré-agrégés. Une attention particulière a été portée à l'alignement temporel entre les différentes sources. Les données Vélib', fournies à une fréquence infra-horaire, ont été agrégées à l'heure pour correspondre à la granularité des données météorologiques. Cette harmonisation est essentielle pour éviter des biais d'asynchronisme dans la modélisation.

2.1.1 Architecture Spark + YARN

Le pipeline repose sur la puissance distribuée du cluster. Le **Master** orchestre les tâches via **YARN**, qui alloue dynamiquement RAM et CPU sur les 3 nœuds (Workers). **Spark** sert de moteur de calcul. Le script Python (Driver) récupère les données, puis Spark distribue l'écriture sur **HDFS** (Data Lake) via les Workers. Cette architecture garantit la tolérance aux pannes et la scalabilité : si un nœud tombe, YARN redistribue le travail, assurant la pérennité des données.

2.1.2 Orchestration : Pourquoi Cron ?

Nous utilisons **Cron**, le planificateur natif Linux, pour sa légèreté et sa fiabilité immédiate. Bien qu'**Apache Airflow** soit le standard industriel (gestion des dépendances, UI), il est trop lourd pour cette infrastructure (nécessite une base de données dédiée et un serveur web). Cron suffit parfaitement pour déclencher nos scripts autonomes à intervalles fixes.

2.1.3 Détail du Workflow ETL

Extraction (Ingestion) Le Driver (Master) interroge les API et stocke la donnée brute sur HDFS partitionnée par date.

— **Vélib (OpenData) :** Dispo temps réel sur Paris, exécuté **toutes les heures**.
— **Météo (Open-Meteo) :** Historique journalier sur Paris, exécuté **toutes les 24h**.

Exemple de Cron + Spark Submit

```
0 * * * * spark-submit --master yarn --deploy-mode client
--num-executors 2 velib_etl.py
```

— `0 * * * *` : Déclenchement à la minute 0 de chaque heure.
— `--master yarn` : Utilise les 3 nœuds du cluster pour le stockage.
— `--deploy-mode client` : Le Driver reste sur le Master (accès Internet pour l'API), tandis que les Executors sur les nœuds écrivent dans HDFS.

Transformation Exécutée **tous les 2 mois**. Spark lit les mois de données brutes (JSON/CSV) depuis HDFS, nettoie les valeurs aberrantes, joint la météo aux données Vélib (Feature Engineering) et sauvegarde le résultat en format optimisé (**Parquet**) dans un dossier HDFS "Processed".

Load (Entraînement) Exécuté **tous les 2 mois**, juste après la transformation. Les données propres sont chargées depuis HDFS vers la mémoire de la machine d'entraînement (ou une machine GPU distante) pour générer le nouveau modèle ML.

2.2 Métriques d'Évaluation de la Qualité des Données

Avant tout traitement, la qualité des données a été évaluée selon plusieurs dimensions. Le taux de complétude excède 99,9 % pour les variables essentielles (`numbikesavailable`, `capacity`), ce qui a permis d'éviter des stratégies d'imputation lourdes. La cohérence temporelle a été vérifiée en comparant les horodatages entre les jeux de données ; quelques incohérences mineures ont été corrigées par alignement sur l'heure la plus proche. La distribution des variables cibles a été analysée via des mesures d'asymétrie et de détection d'outliers (méthode IQR). Enfin, l'intégrité des relations logiques a été testée : par exemple, on a vérifié que la somme des vélos mécaniques (`mechanical`) et électriques (`ebike`) était bien égale au nombre total de vélos disponibles (`numbikesavailable`). Ces contrôles préliminaires ont permis d'identifier quelques erreurs de saisie (valeurs de capacité nulles) et de les corriger de manière appropriée.

2.3 Pipeline de Prétraitement

Le prétraitement a été conçu sous forme de pipeline reproductible, articulé en six étapes séquentielles. Premièrement, le chargement et la consolidation des fichiers multiples ont été réalisés en veillant à préserver l'ordre chronologique. Deuxièmement, un nettoyage a été appliqué pour supprimer les valeurs aberrantes détectées lors de l'analyse exploratoire. Troisièmement, l'ingénierie des caractéristiques temporelles a permis de créer des variables dérivées de l'horodatage (heure, jour, mois, etc.). Quatrièmement, les données météorologiques ont été fusionnées via une jointure sur la clé heure. Cinquièmement, les variables catégorielles (comme le jour de la semaine) ont été encodées, soit en *one-hot encoding*, soit en encodage cyclique pour les variables périodiques. Enfin, une normalisation a été appliquée aux variables numériques à l'aide d'un `StandardScaler`, afin de garantir une échelle commune pour les algorithmes sensibles.

Des décisions techniques importantes ont guidé ce pipeline. La séparation des données en ensembles d'apprentissage et de test (80/20) a respecté la chronologie pour éviter tout *look-ahead bias*. La standardisation a été préférée à la normalisation min-max pour préserver la distribution des données. Enfin, une vigilance particulière a été portée à l'exclusion proactive des variables susceptibles d'introduire un *data leakage*, comme celles dérivées directement des cibles. Cette anticipation s'est révélée cruciale, comme le montrera la suite de l'analyse.

2.4 Ingénierie des Caractéristiques

2.4.1 Caractéristiques Temporelles et Encodage Cyclique

L'ingénierie des caractéristiques temporelles a été menée pour capturer les patterns saisonniers et cycliques inhérents aux données de mobilité. À partir du timestamp exact (`duedate`), l'heure de la journée (0–23) et le mois (1–12) ont été extraits pour modéliser les variations horaires et mensuelles. Le jour de la semaine (0 pour lundi) et un indicateur binaire de weekend ont également été créés pour différencier les jours ouvrables des week-ends.

Cependant, une simple extraction numérique ne suffit pas à représenter la nature circulaire du temps. Pour traiter correctement cette cyclicité, un encodage trigonométrique a été appliqué. L'heure a ainsi été transformée en deux nouvelles variables : `hour_sin` et `hour_cos`, calculées respectivement comme `sin(2 * heure / 24)` et `cos(2 * heure / 24)`. Cette représentation permet au modèle de comprendre que minuit (0) est adjacent à 23h, évitant une discontinuité artificielle. Le même encodage a été appliqué au jour de la semaine, garantissant que dimanche et lundi soient traités comme des valeurs proches.

2.4.2 Caractéristiques Dérivées et Intégration Contextuelle

Plusieurs métriques dérivées ont été calculées pour normaliser l'analyse entre stations de tailles différentes. Les ratios d'occupation `bikes_ratio` (vélos disponibles / capacité totale) et `docks_ratio` (places libres / capacité totale) ont été introduits. Ces ratios standardisés permettent de comparer des stations de capacités variées sur une même échelle. Une correction a été appliquée pour les valeurs de capacité nulles (erreurs potentielles), remplacées par 1 pour éviter toute division par zéro.

L'enrichissement contextuel a été réalisé via l'intégration de données externes. Les données météorologiques horaires (température, précipitations, etc.) ont été jointes sur la base de l'heure. Un indicateur binaire `is_raining` a été créé à partir des données de pluie. Parallèlement, les dates de vacances scolaires de l'académie de Paris ont été récupérées via une API publique, et un indicateur binaire `is_holiday` a été ajouté. Ces variables contextuelles visent à capter des variations de demande liées à des facteurs exogènes.

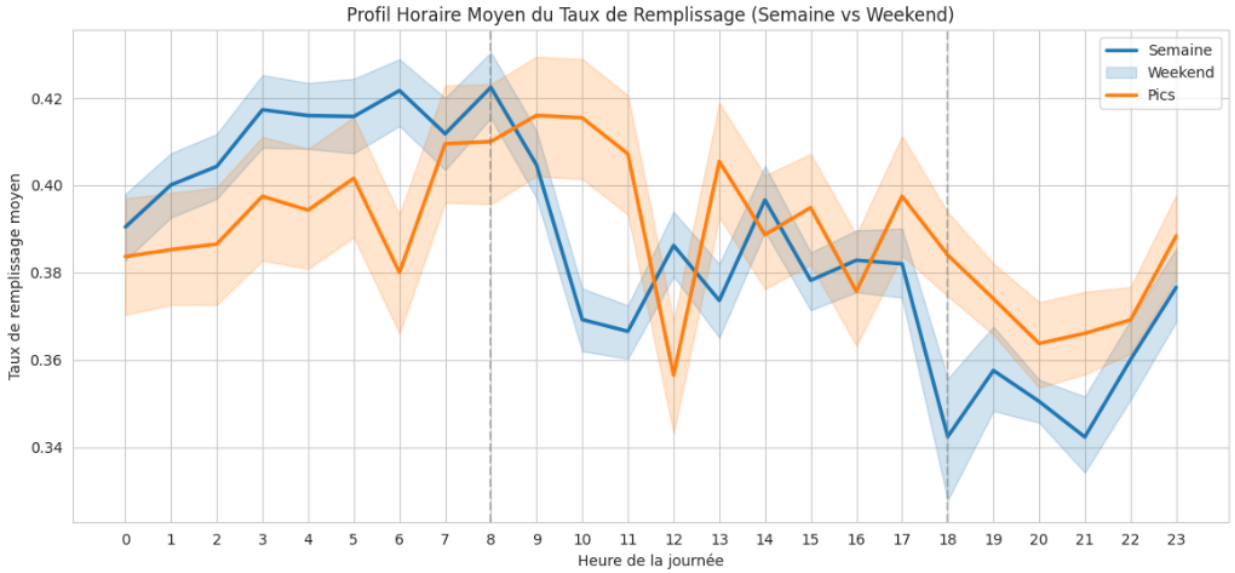


FIGURE 4 – Taux de remplissage moyenne (semaine vs weekend)

2.4.3 Préparation pour la Modélisation

L'ensemble des caractéristiques créées a été conçu pour fournir au modèle des signaux temporels riches (linéaires et cycliques), des métriques normalisées indépendantes de la taille des stations, un contexte externe influençant le comportement des utilisateurs, et des indicateurs catégoriels encodés de manière appropriée. Cette phase d'ingénierie a transformé les données brutes en un jeu de caractéristiques prêt pour l'apprentissage automatique, tout en préservant les relations temporelles et contextuelles essentielles à la prédiction.

3 Stratégie d'Évaluation et de Comparaison des Modèles

3.1 Protocole de Validation Croisée Temporelle

La validation a été réalisée selon une stratégie de validation croisée temporelle à 3 folds, respectant l'ordre chronologique des observations. Cette approche prévient tout *data leakage* temporel et garantit une évaluation réaliste des performances en conditions opérationnelles. La métrique principale retenue est le score R^2, qui mesure la proportion de variance expliquée par le modèle. Des métriques secondaires, telles que l'erreur absolue moyenne (MAE) et la racine de l'erreur quadratique moyenne (RMSE), ont également été calculées pour une interprétation plus opérationnelle.

Le protocole d'évaluation suit une séquence rigoureuse : préparation des features et des cibles (en excluant soigneusement les variables dérivées des cibles), split temporel (141 924 observations pour l'entraînement, 35 482 pour le test), validation croisée sur l'ensemble d'entraînement, sélection du modèle sur la base des performances en CV, optimisation hyperparamétrique via Optuna (50 essais), évaluation finale sur le jeu de test, et analyse d'importance des caractéristiques.

3.2 Benchmark des Algorithmes

Quatre algorithmes ont été comparés de manière systématique : XGBoost (via `MultiOutputRegressor`), LightGBM, CatBoost et Random Forest. Le choix de ces modèles couvre différentes familles d'approches ensemblistes (boosting et bagging) et permet d'évaluer leur robustesse relative. Les critères de comparaison incluent la performance prédictive (R^2, MAE, RMSE), le temps d'entraînement et d'inférence, la robustesse à l'overfitting et l'interprétabilité via l'importance des caractéristiques.

Le pseudo-code suivant résume le protocole d'évaluation :

```
1. Préparation features/targets (exclusion variables dérivées)
2. Split temporel (train: 141,924 obs, test: 35,482 obs)
3. Validation croisée 3-fold sur train set
4. Sélection modèle sur performance CV
5. Optimisation hyperparamétrique (Optuna, 50 essais)
6. Évaluation finale sur test set
```

3.2.1 Performance Initiale et Détection d'une Anomalie

Les premiers résultats, obtenus avant correction d'un *data leakage*, montraient des performances exceptionnellement élevées, comme en témoigne le tableau ci-dessous :

Modèle	R^2 Moyen (CV)	± Écart-type	Temps (s)
XGBoost	**0.9982**	0.0022	6.3
LightGBM	0.9980	0.0023	10.2
CatBoost	0.9978	0.0022	44.9
RandomForest	0.9978	0.0022	684.1

Des scores R^2 supérieurs à 0,99 sont statistiquement improbables pour un problème réel de prédiction de comportement humain dans un système urbain complexe. Cette anomalie a déclenché une investigation approfondie pour identifier la source d'un éventuel *data leakage*.

3.2.2 Investigation et Correction du Data Leakage

L'audit méthodologique a combiné analyse d'importance des caractéristiques (impurity-based), vérification manuelle des dépendances logiques et tests d'ablation. Trois groupes de variables problématiques ont été identifiés : les variables `mechanical` et `ebike`, dont la somme est égale à la cible `numbikesavailable` ; les ratios `bikes_ratio` et `docks_ratio`, calculés directement à partir des cibles ; et la variable `capacity`, fortement corrélée structurellement aux cibles.

La suppression de ces variables a conduit à une chute drastique du R^2, de 0,9982 à 0,4314, confirmant ainsi la présence d'un *data leakage* majeur. Cette correction, bien que douloureuse en termes de performance apparente, a permis d'obtenir un modèle réaliste et généralisable.

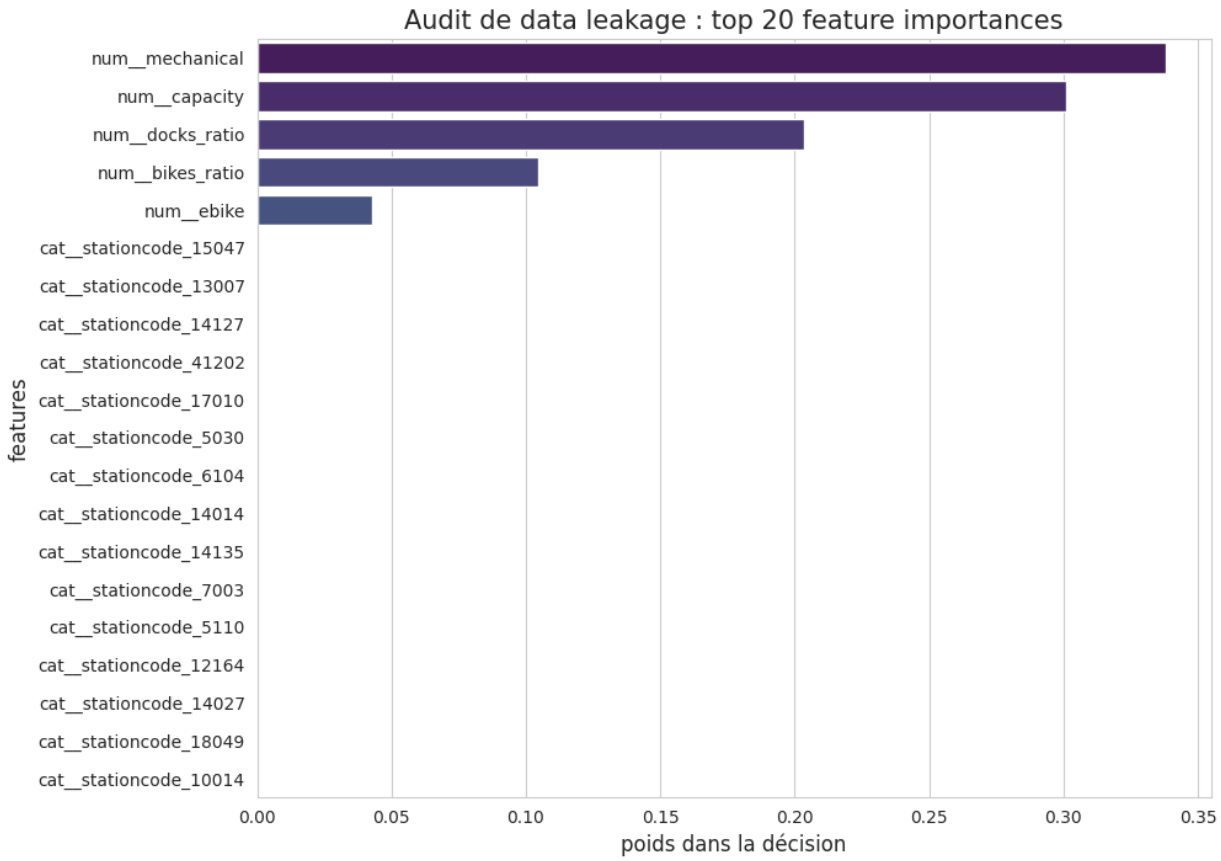


FIGURE 5 – Audit de data leakage

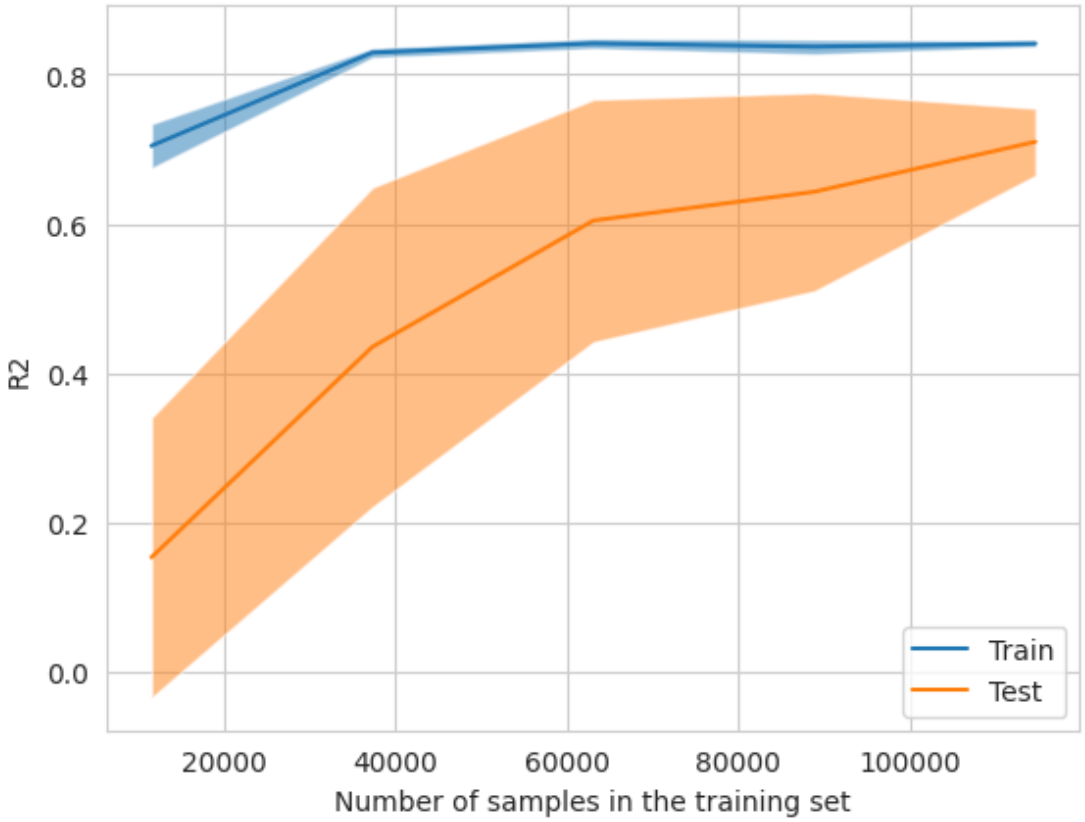
3.2.3 Performance réelle post-correction et optimisation

Après nettoyage des variables contaminées, les performances sont devenues réalistes mais modestes. Le modèle XGBoost de base atteignait un R^2 de 0,4314, une MAE de 6,12 et une RMSE de 7,89. Une optimisation hyperparamétrique poussée via Optuna (50 itérations, recherche bayésienne TPE) a permis d'améliorer significativement ces métriques, comme le montre le tableau suivant :

Métrique	Avant Optimisation	Après Optimisation	Amélioration
R^2	0.4314	0.7098	**+64,5%**
MAE	6.12	4.6	-24,8%
RMSE	7.89	6.26	-20,7%

Les hyperparamètres optimisés incluent un `learning_rate` de 0,2994, une `max_depth` de 10, un `reg_alpha` de 40,3, et un `n_estimators` de 663, entre autres. Cette optimisation a permis au modèle d'exploiter au maximum le signal disponible dans les données tout en contrôlant le risque de sur-apprentissage.

```
R2 0.7098308205604553
MSE 39.24848175048828
RMSE 6.264860872396791
MAE 4.603778839111328
```



4 Bilan des difficultés rencontrées et solutions apportées

4.1 Data Leakage

La difficulté majeure a été la présence insidieuse de variables dérivées des cibles, conduisant à des performances artificiellement parfaites. La solution a reposé sur un audit systématique des caractéristiques, une vérification manuelle des relations logiques et des tests d'ablation. Cette expérience souligne l'importance d'une vigilance méthodologique constante et d'une analyse critique des performances trop élevées.

4.2 Complexité du problème réel

La prédiction de comportements humains dans un système urbain est intrinsèquement complexe et stochastique. De nombreux facteurs externes (événements locaux, travaux, interactions spatiales entre stations) n'ont pas été capturés dans les données. Pour atténuer cette limite, un *feature engineering* temporel avancé (encodage cyclique) et l'intégration de données contextuelles (météo, vacances) ont été mis en œuvre. Le modèle retenu est un modèle multi-cibles qui capture partiellement les corrélations entre les sorties. Néanmoins, un plafond de performance persiste, reflétant la complexité inhérente du système.

4.3 Volumétrie et hétérogénéité des données

La gestion de 1 524 fichiers sources, dans des formats multiples (Parquet, JSON, CSV) et à des fréquences temporelles différentes, a représenté un défi technique. Un pipeline modulaire a été développé, avec un traitement spécifique par format, un alignement temporel rigoureux sur une base horaire, une persistance intermédiaire des données en Parquet pour assurer la reproductibilité, et des vérifications de cohérence à chaque étape d'intégration.

4.4 Choix et évaluation des modèles

La comparaison équitable de modèles aux caractéristiques différentes a nécessité un benchmark standardisé avec les mêmes données et métriques. Une pré-sélection basée sur le compromis performance/temps de calcul a permis de focaliser l'optimisation sur les algorithmes les plus prometteurs (XGBoost, LightGBM). La validation temporelle et l'optimisation hyperparamétrique ciblée ont finalement conduit au choix de XGBoost comme modèle optimal pour ce projet.

5 Conclusions et recommandations

5.1 Synthèse des apprentissages

Ce projet a mis en lumière plusieurs apprentissages clés. Premièrement, une vigilance méthodologique constante est nécessaire : un score trop parfait doit toujours alerter et déclencher une investigation. Deuxièmement, l'audit systématique des caractéristiques, via l'analyse d'importance, est une étape non-négociable pour détecter les fuites d'information. Troisièmement, la transparence dans la documentation des problèmes rencontrés et des corrections apportées renforce la crédibilité scientifique. Enfin, il est crucial d'accepter les limites inhérentes aux problèmes complexes et d'interpréter les performances avec réalisme.

5.2 Évaluation du modèle final

Le modèle final (XGBoost optimisé) présente des points forts notables : une performance réaliste ($R^2 = 0{,}71$) sur un problème difficile, une robustesse assurée par la régularisation et la validation temporelle, un temps d'inférence court compatible avec une application en temps réel, et une interprétabilité partielle via l'importance des caractéristiques. Ses principales limitations incluent l'absence de modélisation des interactions spatiales entre stations, le manque de *features* retardées (*lag features*) pour capter l'historique immédiat, la limitation des variables contextuelles (pas d'événements locaux par exemple), et le caractère statique du modèle sans mécanisme d'adaptation continue dans le temps.

5.3 Pistes d'amélioration

À court terme, des améliorations sont envisageables avec les données actuelles : ajout de *lag features* (t-1, t-2), regroupement préalable des stations similaires via du clustering, et développement de modèles spécifiques par type de station ou plage horaire. À moyen terme, l'enrichissement par des données externes (trafic, événements, données de transports en commun) serait précieux. Une architecture hiérarchique (prédiction agrégée puis désagrégée) ou une approche spatio-temporelle explicite pourraient également permettre de mieux capter les dynamiques du système.

Le modèle constitue une base solide pour des travaux ultérieurs et sert de cas d'étude instructif sur les pièges méthodologiques en science des données. Sa performance, bien que perfectible, est suffisante pour des applications de planification grossière, tout en identifiant clairement les voies d'amélioration pour une gestion plus fine et plus réactive du système de vélos en libre-service.

Conclusion

Au terme de ce projet, le bilan dépasse la simple obtention d'un score de prédiction. Ce challenge nous a permis d'appréhender la complexité systémique du Big Data, où la performance du code est indissociable de la robustesse de l'infrastructure qui le porte. Nous avons réussi à transformer deux ordinateurs standards en un cluster de calcul distribué fonctionnel, capable de traiter des volumes de données qui auraient saturé une machine unique.

Sur le plan technique, la maîtrise de l'Infrastructure as Code (IaC) via **Ansible** a été le facteur clé de notre réussite opérationnelle. Elle a transformé une tâche d'installation fastidieuse et propice aux erreurs en un processus industriel et itératif. Cette couche d'automatisation a libéré du temps précieux pour la phase d'analyse de données, confirmant l'adage selon lequel "80% du travail d'un Data Engineer se situe dans la préparation".

Sur le plan analytique, la confrontation avec le **Data Leakage** restera l'enseignement majeur de ce semestre. L'obtention initiale de scores R^2 quasi-parfaits (>0.99), suivie de l'investigation rigoureuse qui a révélé la fuite d'information, nous a rappelé qu'en Science des Données, un résultat "trop beau pour être vrai" est souvent synonyme d'erreur méthodologique. Le modèle final (XGBoost), avec un R^2 de 0.71, bien que moins spectaculaire en apparence, est infiniment plus précieux car il est réaliste, généralisable et robuste face aux données futures.

Pour aller plus loin, plusieurs perspectives d'industrialisation s'ouvrent. L'architecture actuelle, basée sur des traitements par lots (Batch), pourrait évoluer vers du temps réel via **Spark Streaming** pour ajuster les prédictions à la minute près. De plus, l'intégration d'une couche **MLOps** avec un outil comme MLflow permettrait de surveiller la dérive du modèle (Drift) en production. Enfin, la migration de nos scripts Ansible vers un fournisseur Cloud (AWS ou Azure) permettrait de lever les contraintes matérielles que nous avons dû contourner, offrant une scalabilité horizontale immédiate.

Ce projet a validé notre capacité à orchestrer des technologies hétérogènes (Linux, Hadoop, Spark, Python) pour répondre à une problématique métier concrète, nous préparant efficacement aux exigences des environnements de production en entreprise.